

Smart Hire

Architecture Overview

An event-driven, microservices-based AI recruitment platform

Project Summary

Smart Hire is an event-driven recruitment platform designed as six cooperating microservices. It supports recruiter and candidate workflows, AI-assisted matching, multi-stage application processing, notifications, analytics, and operational monitoring.

The architecture separates business responsibilities and data ownership across services. Each transactional service owns its own database and exposes APIs or events rather than sharing database tables directly.

The Six Microservices

1

User Service

Identity and profile management layer

Responsibilities

- User registration and account management
- Login and JWT token creation
- Token revocation during logout
- Role-based access control for candidates and recruiters
- Candidate profile management
- Resume uploads and storage in MinIO
- Publishing user and candidate events for downstream consumers

Owns: User accounts, roles, candidate profiles, authentication data, and token records.

2

Job Service

Owns the recruitment domain's transactional data

Responsibilities

- Creating and managing job postings
- Validating recruiter ownership
- Publishing jobs
- Defining interview and screening stages
- Receiving candidate applications
- Tracking application and stage status
- Storing AI match scores on applications
- Publishing business events through Kafka

Owns: Jobs, applications, job stages, application stages, statuses, rejection reasons, and outbox records.

3

Workflow Service

The orchestration layer for long-running processes

Responsibilities

- Coordinating calls between the Job, User, and AI services
- Managing workflow state transitions
- Retrying transient failures
- Delegating AI-heavy work to AI Service activities
- Waiting for recruiter decisions using Temporal signals
- Handling candidate withdrawal and stage timeouts
- Producing successful and failed workflow outcomes

It runs three main workflow types: job publishing, candidate profile processing, and application processing. It coordinates these processes but does not own the core business data.

4

AI Service

Intelligent processing and matching capabilities

Responsibilities

- Extracting structured information from resumes
- Extracting structured information from job descriptions
- Creating candidate and job AI profiles
- Generating semantic embeddings
- Storing embeddings with PostgreSQL and pgvector
- Calculating candidate-to-job skill match scores
- Providing an AI assistant for recruiters and candidates
- Publishing AI usage and processing events

Owns: Groq is used for language-model processing, Hugging Face models are used for embeddings, and MinIO is used to retrieve resume files.

5

Analytics Service

Reporting and operational insight layer

Responsibilities

- Consuming events from Kafka
- Persisting event envelopes and payloads
- Tracking failed workflow events separately
- Calculating hiring funnel and platform metrics
- Providing recruiter dashboard APIs
- Tracking AI assistant usage
- Exposing health and Prometheus metrics endpoints

Owns: Not the system of record for hiring operations; it builds its view of the platform from events emitted by other services.

6

Notification Service

Event-driven communication worker

Responsibilities

- Consuming application lifecycle events from Kafka
- Rendering candidate-facing email templates
- Sending emails through SMTP
- Notifying candidates when applications are received
- Notifying candidates about stage progress, rejection, withdrawal, and hiring
- Acknowledging Kafka messages only after successful processing

Owns: Has a deliberately small HTTP surface because its main responsibility is asynchronous event processing.

Typical Application Flow

- 1 A candidate submits an application to the Job Service.
- 2 The Job Service validates the candidate and job, creates the application, and returns control to the client.
- 3 The Job Service starts an application-processing Temporal workflow.
- 4 The Workflow Service fetches candidate details from the User Service.
- 5 The workflow asks the AI Service to calculate a skill match score.
- 6 If the candidate has no AI profile, a child candidate workflow generates one from the resume.
- 7 The workflow retries the skill match operation.
- 8 The score is stored in the Job Service.
- 9 Applications below the configured threshold are automatically rejected.
- 10 Passing applications receive job-specific application stages.
- 11 The workflow waits for recruiter approval, rejection, candidate withdrawal, or a stage timeout.
- 12 The application eventually becomes rejected, withdrawn, hired, or workflow-failed.
- 13 Status changes create outbox events.
- 14 The outbox worker publishes those events to Kafka.
- 15 Analytics stores the events and Notification sends relevant candidate emails.

Why Kafka Was Introduced

Kafka is used for asynchronous communication and event distribution.

Examples of published events include:

```
CandidateCreated | JobPublished | ApplicationSubmitted | ApplicationStageCompleted  
| CandidateRejected | CandidateWithdrawn | CandidateHired | WorkflowFailed
```

Kafka was introduced for several reasons:

- It decouples core hiring transactions from analytics and notification delivery.
- Multiple consumers can react to the same event independently.
- Analytics and notification processing can be scaled separately.
- Temporary failure in a downstream consumer does not have to block the hiring workflow.
- New consumers can be added without changing the original producer.
- Events provide a useful history of business activity.

Transactional Outbox

The Job Service uses a transactional outbox pattern. A business state change and its corresponding event are written to the same database transaction.

A background worker then:

- 1 Reads unpublished outbox records.

- 2 Publishes the events to Kafka.
- 3 Marks the records as published after publication succeeds.

This prevents a state change from succeeding while its event is lost because the service crashed before publishing.

Why Temporal Was Introduced

Kafka distributes events, but it does not coordinate a long-running process with dependent steps, timers, retries, and human decisions. Temporal fills that orchestration role.

Application processing can involve:

- Multiple service calls
- AI and external-provider calls
- Database updates
- Automatic rejection rules
- Recruiter decisions that may arrive hours or days later
- Candidate withdrawal
- Stage timeouts
- Recovery after worker or service crashes

Temporal provides:

- Durable workflow state
- Activity retry policies
- Timers and timeout handling
- Signals for human decisions
- Child workflows for candidate profile generation
- Deterministic workflow IDs
- The ability to resume work after a worker restarts

The design separates deterministic workflow logic from side-effecting activities. Workflows decide what should happen next. Activities perform HTTP calls, database updates, and AI processing.

What the AI Does

The AI features are focused on document understanding, semantic matching, and conversational assistance (implemented, though not yet using RAG over the stored embeddings). They are not intended to make an autonomous hiring decision.

Resume and Job Understanding

For a candidate resume, the AI pipeline:

- 1 Downloads the PDF or DOCX resume from MinIO.
- 2 Extracts its text.
- 3 Sends the text to Groq for structured extraction of skills, experience, education, certifications, and years of experience.
- 4 Combines the extracted information with the source text.
- 5 Generates a vector embedding and stores the candidate AI profile.

For a job description, the pipeline similarly extracts skills, keywords, requirements, responsibilities, benefits, and experience level before creating a job AI profile and embedding.

Models and APIs

- **Groq API:** Used for language-model tasks such as extracting structured resume and job details. The documented model is llama-3.3-70b-versatile.
- **Hugging Face / Sentence Transformers:** Used to generate semantic embeddings. The current design uses 384-dimensional vectors.
- **PostgreSQL with pgvector:** Stores and searches the embeddings.
- **MinIO:** Stores and serves candidate resume files; it is storage, not an AI model.

Matching and Retrieval

Candidate-job matching compares the candidate and job embeddings using cosine similarity and converts the result into a 0-100 skill match score. The application workflow uses a configured threshold, currently 50, as one screening rule.

The AI assistant answers recruiter or candidate questions using context gathered from direct lookups against the Job, User, and AI services (job details, job/candidate AI profiles, application data). This context-gathering step is implemented today. Using vector similarity search over the stored embeddings to retrieve that context, rather than direct lookups, is not yet implemented; see Current Implementation Status below.

Is It Agentic?

The product does not currently implement a fully autonomous agent loop. The AI does not independently plan multi-step tasks, select arbitrary tools, or make hiring decisions without workflow rules and human approval.

The surrounding Temporal workflows are deterministic orchestration, not AI agents. They coordinate predefined activities such as extracting text, generating embeddings, calculating a score, updating status, and waiting for

recruiter signals. Recruiters remain responsible for human approval stages.

Current Implementation Status

Implemented: Resume chunking, resume embeddings, and job embeddings, the storage layer (pgvector) needed to hold and query them, and the AI assistant itself, intent detection, permission checks, direct-lookup context building, prompt building, and Groq-generated responses.

Not yet implemented: RAG (retrieval-augmented generation), using vector similarity search over the stored embeddings to pull relevant context into the assistant's prompt. The assistant currently gathers context through direct, rule-based lookups instead. Adding that retrieval layer is the next goal currently in progress.

Output Quality and Limitations

Current quality controls include:

- Input validation for job descriptions and extracted resume text
- Structured extraction into defined profile fields
- Embedding dimension and persistence checks
- Cosine-similarity scoring rather than keyword matching alone
- A visible match score and explicit workflow threshold
- Retry policies for temporary Groq, Hugging Face, storage, and database failures
- Workflow and event telemetry for diagnosing failed processing

Failure and Retry Handling

Temporal Activity Retries

Activities that call other services or external providers can fail transiently. Temporal retries operations such as:

- Updating job and application status
- Creating application stages
- Fetching job stages or user details
- Calling the AI Service
- Calling Groq
- Generating embeddings
- Saving AI profiles

Different activities can use different retry limits. External LLM calls generally use fewer attempts than database operations.

Deterministic Workflow IDs

Job publishing and application processing use IDs derived from the entity being processed, such as:

```
job-publishing-workflow-{job_id}  
application-processing-workflow-{application_id}
```

This prevents duplicate workflows when an API request is repeated. If a workflow-start call appears to fail, the system can check whether the workflow already exists and treat the result as an idempotent success.

Explicit Failure States

Failures are represented in business data instead of being hidden in logs. Examples include:

- FAILED
- WORKFLOW_FAILED
- WorkflowStartFailed
- WorkflowRetryExhausted
- Failed or withdrawn application stages

This gives recruiters and operators visibility into incomplete processing and supports recovery decisions.

Kafka Producer Reliability

Kafka producers are configured with:

- acks=all, so the broker confirms the message safely
- Producer retries
- Delivery callbacks for logging failures
- Trace headers for cross-service correlation

The transactional outbox remains the main protection against losing business events when database and message publication happen at different times.

Kafka Consumer Reliability

Analytics and Notification use manual offset commits:

- 1 Poll a message.
- 2 Decode and process it.
- 3 Persist the result or send the email.
- 4 Commit the offset only after successful processing.

If processing fails before the commit, Kafka can deliver the message again. Analytics also uses the event ID as an idempotency key so duplicate deliveries do not distort metrics.

Human-in-the-Loop Recovery

Temporal signals allow the application workflow to wait safely for recruiter decisions. Timers handle stages that exceed their allowed duration. The workflow can then apply the appropriate rejection or timeout outcome without keeping an HTTP request open.

Architectural Decisions and Reasons

Separate Databases per Service

Each service owns its data. This reduces coupling, protects service boundaries, and allows the data model of one service to evolve without breaking another service.

HTTP for Immediate Commands and Queries

Synchronous HTTP is used where the caller needs an immediate response, such as authentication, job validation, application creation, or retrieving user details.

Kafka for Events

Kafka is used where the producer should not wait for downstream work, such as analytics ingestion, candidate email delivery, and operational event tracking.

Temporal for Process Orchestration

Temporal is used for multi-step processes that require durable state, retries, timers, signals, and recovery.

Outbox for Reliable Event Publication

The outbox pattern links a database state change to its event reliably and avoids the dual-write problem.

Idempotency for Safe Retries

Retries can repeat requests. Deterministic workflow IDs and event IDs ensure repeated operations do not create duplicate workflows or duplicate analytics records.

Observability as a Design Requirement

OpenTelemetry, Jaeger, Prometheus, and structured logging are used to understand request flow, activity retries, Kafka processing, external dependency failures, and service health.

Technology Used

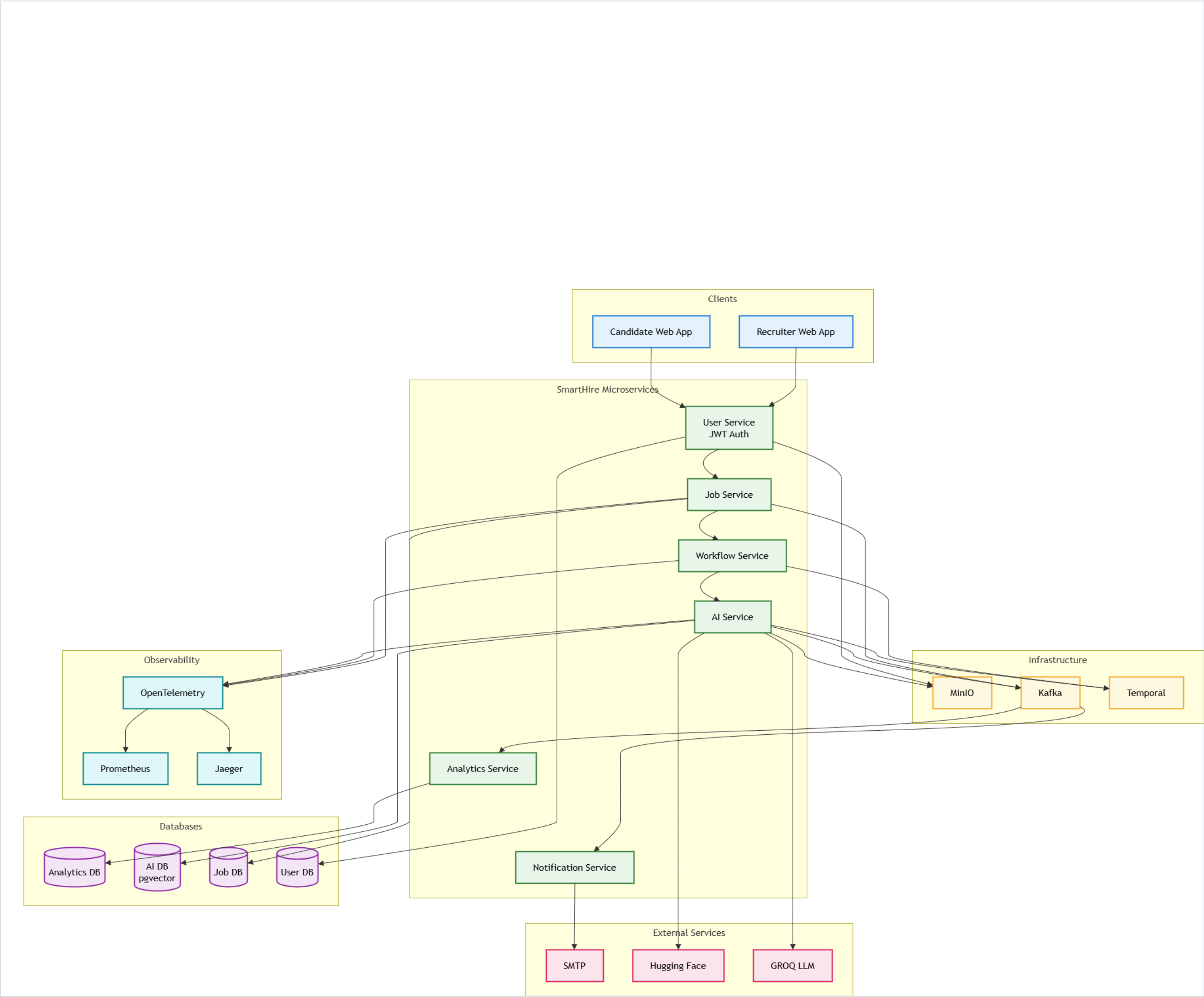
- Python and FastAPI
- PostgreSQL and SQLAlchemy
- Alembic migrations
- Kafka with Confluent Kafka clients
- Temporal Python SDK
- Groq LLM
- Hugging Face embeddings
- PostgreSQL pgvector
- MinIO object storage
- OpenTelemetry, Jaeger, Prometheus
- Docker Compose

Appendix: System Diagrams

Architecture, data model, and sequence diagrams supporting the sections above.

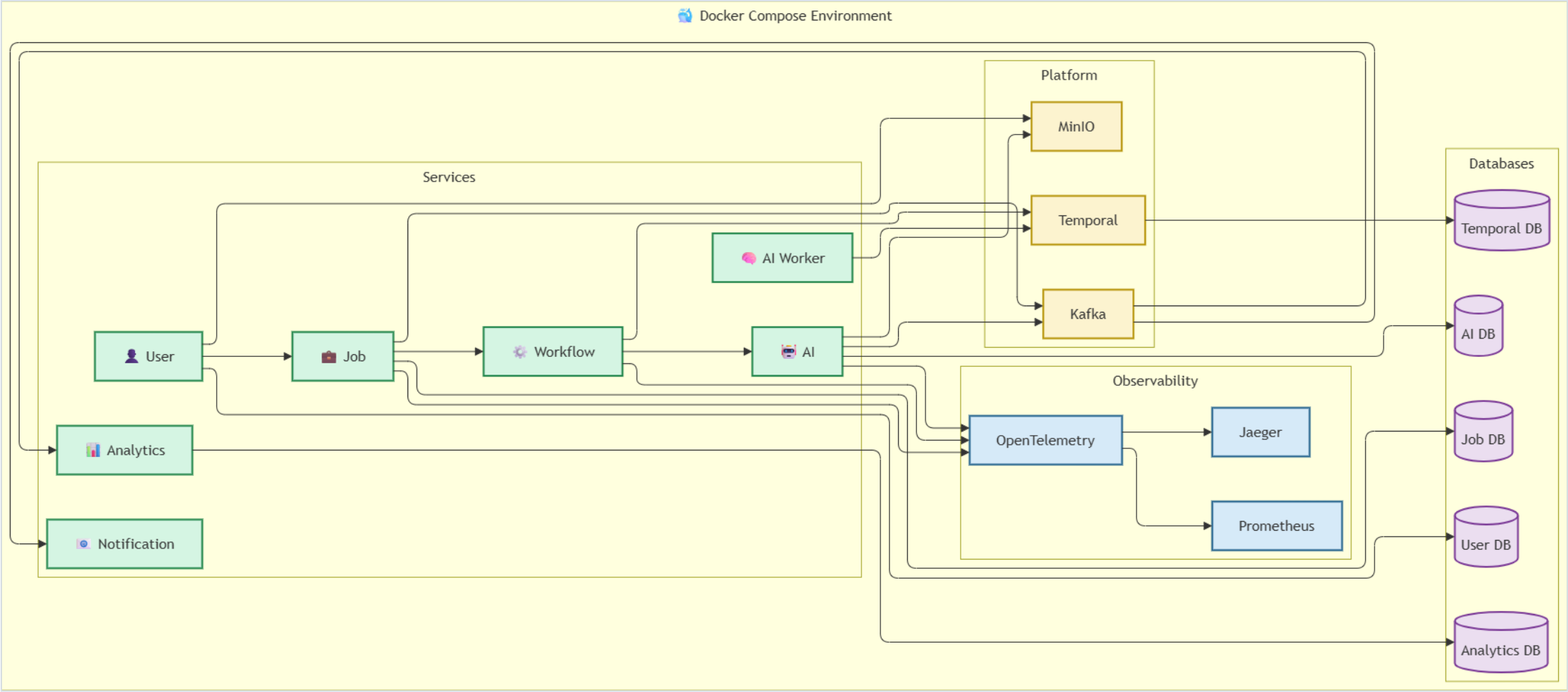
This appendix includes:

- System Architecture
- Docker Compose Deployment
- Entity-Relationship Diagram
- Login - Sequence Diagram
- Job Creation - Sequence Diagram
- Job Publishing & AI Processing - Sequence Diagram
- Candidate Resume AI Processing - Sequence Diagram
- AI Assistant - Sequence Diagram (Planned)



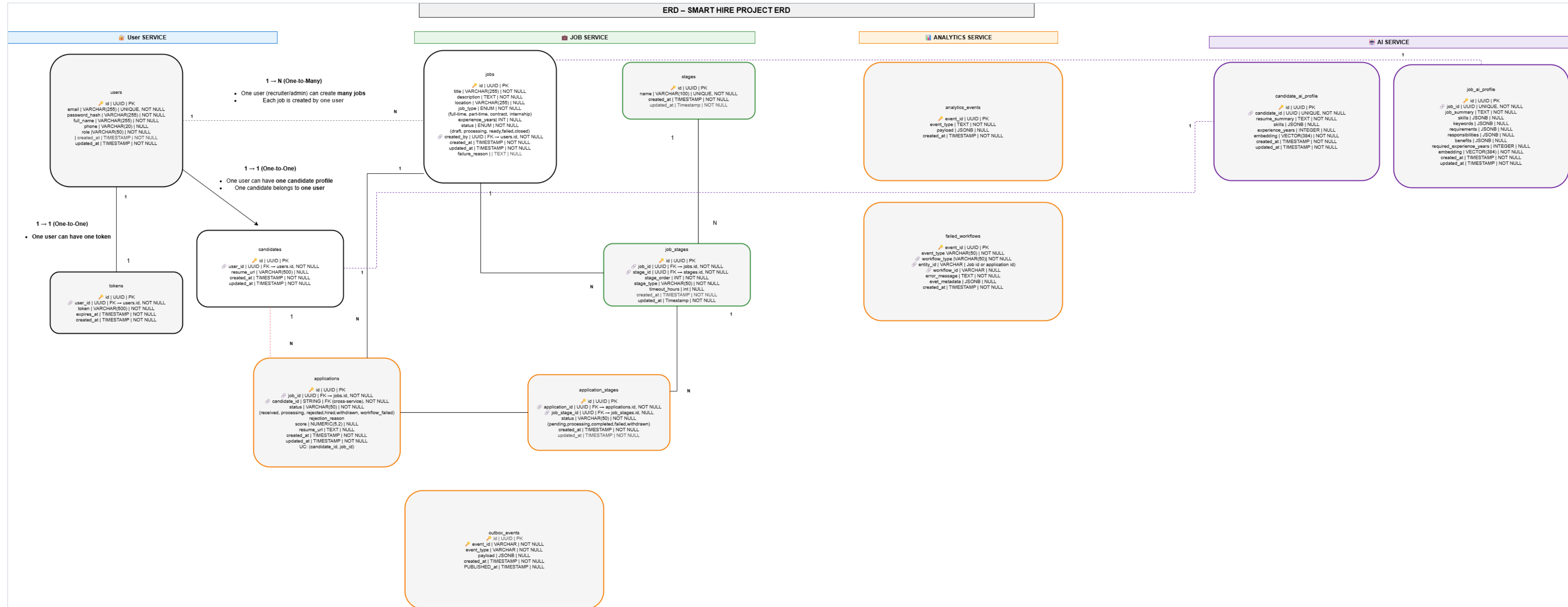
Docker Compose Deployment

How the services, platform components, and databases are containerized and connected.



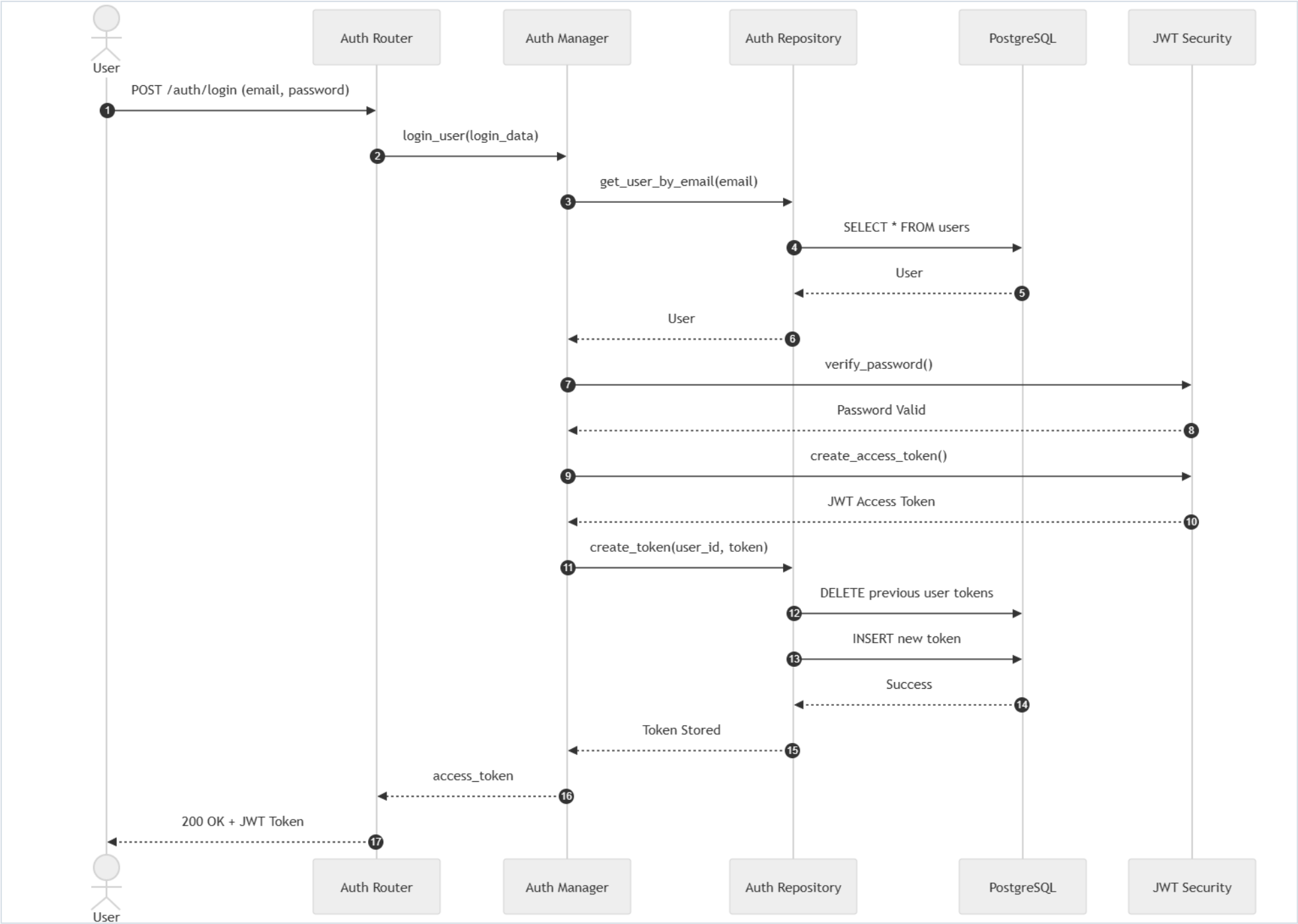
Data ownership and relationships across the User, Job, Analytics, and AI services.

Data ownership and relationships across the User, Job, Analytics, and AI services.



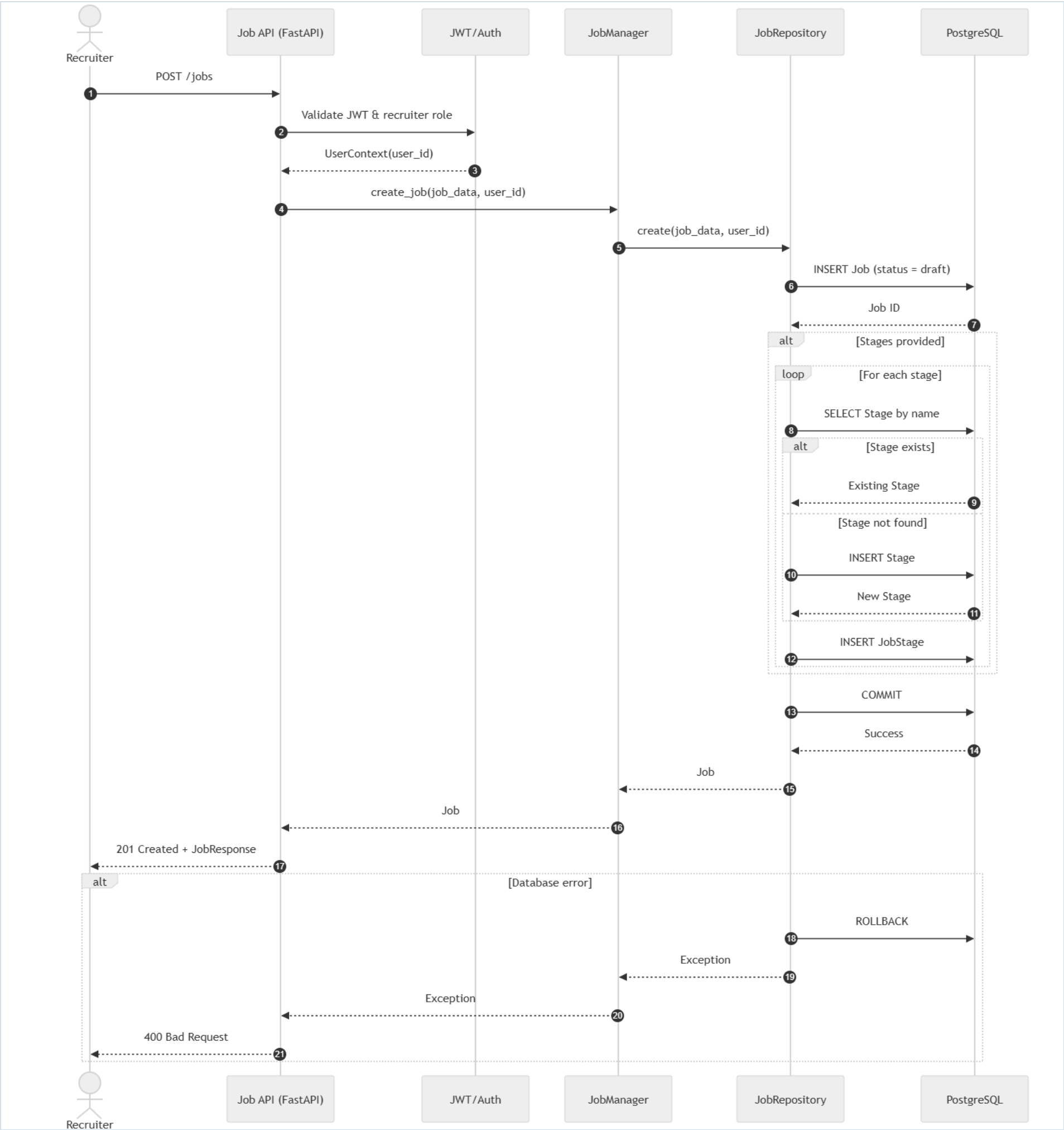
Login - Sequence Diagram

JWT authentication flow from credential submission to token issuance.



Job Creation - Sequence Diagram

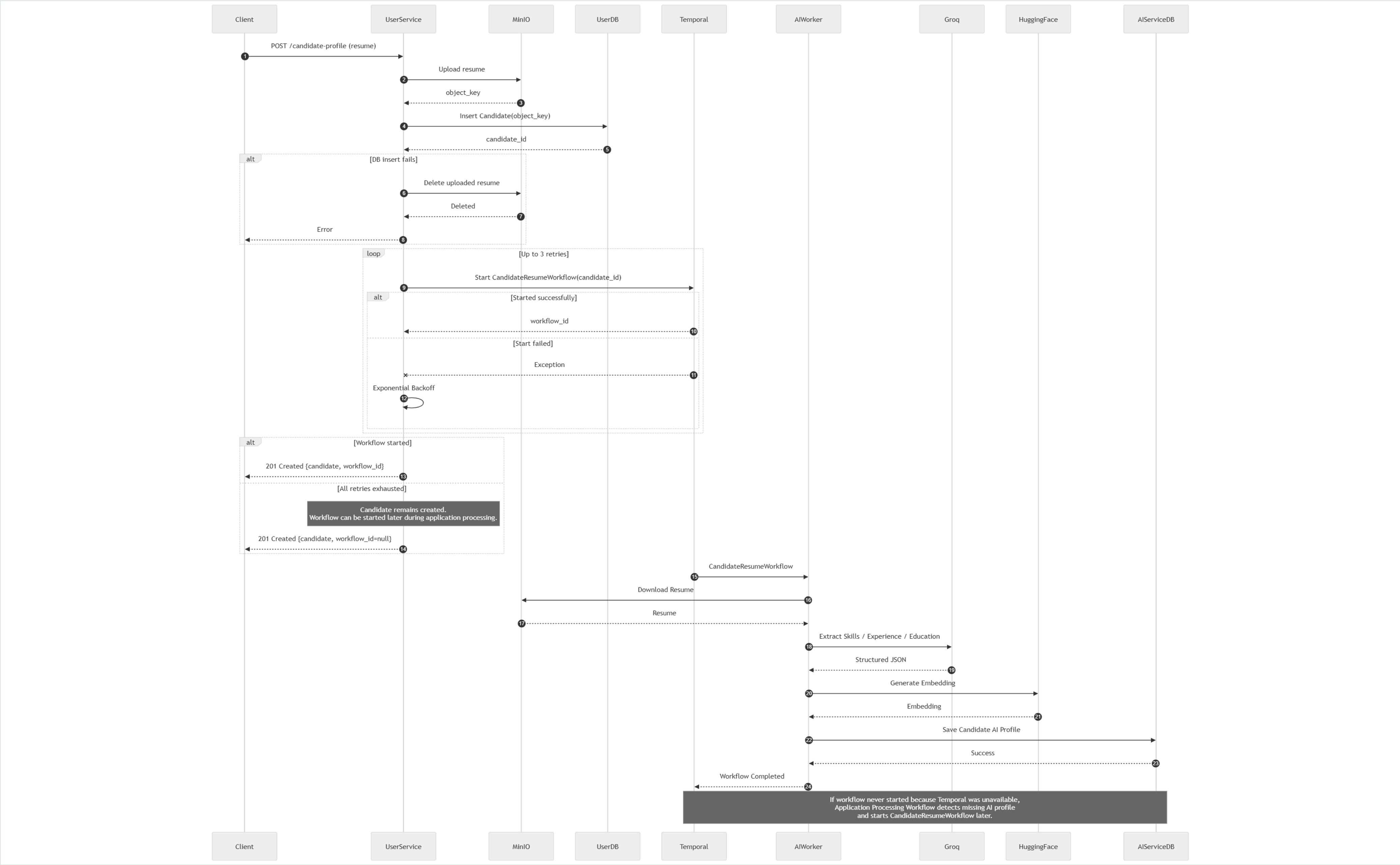
Recruiter job posting flow, including stage creation and transactional rollback on failure.



Publishing workflow, Temporal-orchestrated AI detail extraction and embedding, and failure/retry paths.

Candidate Resume AI Processing - Sequence Diagram

Resume upload through Temporal workflow, extraction, embedding, and profile persistence.



AI Assistant - Sequence Diagram (Planned)

Intended retrieval-augmented flow for the AI assistant: intent detection, context building, and LLM response generation. Not yet implemented.

